

ALGORYTMY PRZETWARZANIA OBRAZÓW

Lab 4
Łukasz Roszkowiak

Segmentacja (adaptacyjna & Otsu)

Zadanie

Opracować algorytm i uruchomić aplikację realizującą segmentację obrazów 3 metodami:

- Ręcznie (interakcyjnie) wyznaczony próg (*thresholding*)
- Progowanie adaptacyjne (*adaptive thresholding*)
- Progowanie metodą Otsu

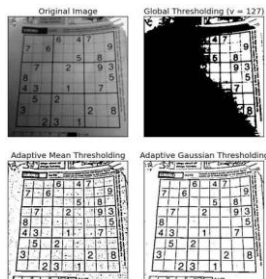
Thresholding

$$q = \begin{cases} L_{\min} & \text{dla } p \leq p_1 \\ L_{\max} & \text{dla } p > p_1 \end{cases}$$

- Wartości powyżej ustalonego progu p_1 przyjmują wartość 1 (lub 255), a wartości poniżej progu przyjmują wartość 0.
- **UWAGA!** Należy pamiętać aby w programach, umożliwić użytkownikowi:
 - interaktywny dobór progu (najlepiej z aktywnym podglądem)
 - konwersję obrazu kolorowego do szaroodcieniowego

Progowanie adaptacyjne

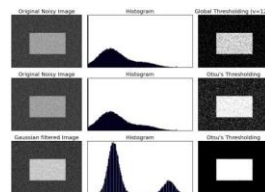
- lokalne progowanie,
- próg wyznaczany na podstawie otoczenia
- schemat okna przesuwanego



*Image source: https://docs.opencv.org/master/d7/d4d/tutorial_py_thresholding.html

Metoda Otsu

- Automatyczny dobór wartości progu
- Skuteczny przy obrazach o histogramie bimodalnym
- Progowanie globalne



*Image source: https://docs.opencv.org/master/d7/d4d/tutorial_py_thresholding.html

Segmentacja GrabCut

Zadanie

Opracować algorytm i uruchomić aplikację realizującą segmentację metodą GrabCut. Aplikację proszę przetestować na obrazach, zawierających obiekt na jednolitym tle.

"GrabCut" — Interactive Foreground Extraction using Iterated Graph Cuts

Carsten Rother¹

Vladimir Kolmogorov²
Microsoft Research Cambridge, UK

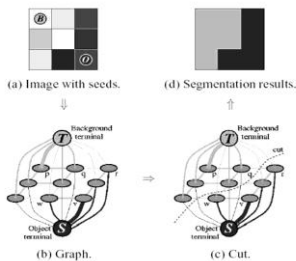
Andrew Blake³



Figure 1: Three examples of GrabCut. The user drags a rectangle loosely around an object. The object is then extracted automatically.

<https://doi.org/10.1145/1015706.1015720>

GrabCut – zasada działania



<https://www.cs.ru.ac.za/research/g02m1682/>

GrabCut - przykład

- Wydobycie obiektu („foreground”) przez usunięcie tła („background”)



GrabCut – OpenCV (1)

- `cv2.grabCut(image, mask, rectangle, backgroundModel, foregroundModel, num_iter, cv2.GC_INIT_WITH_RECT)`
 - `image` – obraz wejściowy
 - `mask` – maska wyjściowa (zera; rozmiar jak `image`)
 - `rectangle` – bounding box obiektu zainteresowania
 - `backgroundModel` – `np.zeros((1, 65), np.float64)`
 - `num_iter` – iteracje metody (przykładowo 3)
 - `cv2.GC_INIT_WITH_RECT` – stała z opencv ponieważ używamy `rectangle`

GrabCut – OpenCV (2)

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('messi5.jpg')
assert img is not None, "file could not be loaded"
mask = np.zeros(img.shape[:2], np.uint8)

bgdModel = np.zeros((1,65), np.float64)
fgdModel = np.zeros((1,65), np.float64)

rect = (50,50,450,290)
cv.grabCut(img,mask,rect,bgdModel,fgdModel,5,cv.GC_INIT_WITH_RECT)

mask2 = np.where((mask==2)|(mask==0),0,1).astype('uint8')
img = img*mask2[:, :, np.newaxis]
plt.imshow(img),plt.colorbar(),plt.show()
```



GrabCut – w programie

- Użytkownik podaje:
 - obraz wejściowy - image
 - prostokąt zawierający obiekt zainteresowania (startingPoint_x, startingPoint_y, width, height) – rectangle
 - albo jako rysowany na obrazie
 - albo jako obraz maski

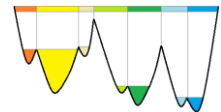
Algorytm Watershed

Zadanie

Opracować algorytm i uruchomić aplikację realizującą segmentację metodą wododziałową (*watershed*). Aplikację proszę przetestować na obrazach, zawierających połączone („sklejone”) obiekty.

Watershed

- Jeśli za wysokość terenu przyjmuje się intensywność pikseli, to za wododział – grzbiety tych funkcji.
- Obszary o małej intensywności stanowią w obrazie lokalne „doliny”
- Obszary o dużej intensywności – lokalne „wzniesienia”



*[image source: <https://media.aneiv-vanity.com/render-output/4644354/figures/wsl.png>]

Metoda Inpainting

Zadanie

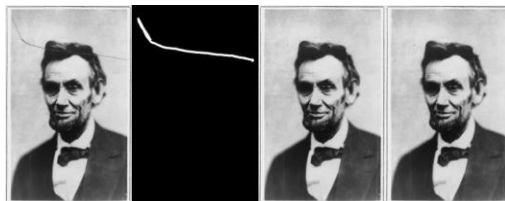
Opracować algorytm i uruchomić aplikację realizującą metodę Inpainting. Aplikację proszę przetestować na obrazach, zawierających pojedyncze linie/odcinki w kolorze czarnym lub białym naniesione na obraz.

Inpainting - przykład



<https://www.geeksforgeeks.org/image-inpainting-using-opencv/>

Inpainting – przykład 2



<https://learnopencv.com/image-inpainting-with-opencv-c-python/>

Podstawy Inpainting

- Dwie publikacje naukowe:
- “An Image Inpainting Technique Based on the Fast Marching Method”, Alexandru Telea, 2004, <https://doi.org/10.1109/CVPR.2001.990497>
- “Navier-Stokes, Fluid Dynamics, and Image and Video Inpainting”, Bertalmio, Marcelo, Andrea L. Bertozzi, and Guillermo Sapiro, 2001, <http://dx.doi.org/10.1080/10867651.2004.10487596>

Inpainting - OpenCV

- `dst = cv2.inpaint(img, mask, iter, cv2.INPAINT_TELEA)`
 - `img` – obraz wejściowy
 - `mask` – maska zawierająca obszary do pokrycia
 - `iter` – iteracje metody (przykładowo 3)
 - stała odpowiedzialna za użytą metodę:
 - `cv2.INPAINT_TELEA`
 - `cv2.INPAINT_NS`

Inpainting – w programie

- Użytkownik podaje:
 - obraz wejściowy – `image`
 - maskę obszarów do uzupełnienia

```
1 import numpy as np
2 import cv2
3
4 # Open the image.
5 img = cv2.imread('cat_damaged.png')
6
7 # Load the mask.
8 mask = cv2.imread('cat_mask.png', 0)
9
10 # Inpaint.
11 dst = cv2.inpaint(img, mask, 3, cv2.INPAINT_NS)
12
13 # Write the output.
14 cv2.imwrite('cat_inpainted.png', dst)
```

Kompresja RLE

Zadanie

Opracować algorytm i uruchomić aplikację realizującą kompresję obrazów metodą RLE:

- Zapis obrazu:
 - bez kompresji
 - po kompresji
- Obliczyć stopień kompresji (SK)

Kompresja

Cel:

- Archiwizacja
- Przesyłanie

Rodzaje:

- bezstratna (*lossless compression*)
- stratna (*lossy compression*)

Example of Lossy Compression



Original Lena Image
(12KB size)

Lena Image,
Compressed (85%
less information,
1.8KB)

Lena Image, Highly
Compressed (96%
less information,
0.56KB)

* Image source: <https://tyletrowenblog.files.wordpress.com/2016/06/example-of-lossy-compression.png>

Stopień kompresji

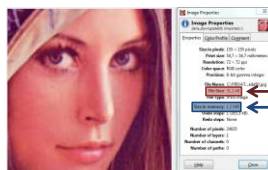
$$SK = \frac{KP}{KW}$$

SK - stopień kompresji krzywej.

KP [bajt] - obszar pamięci zajmowany przez **kod pierwotny** krzywej.

KW [bajt] - obszar pamięci zajmowany przez **kod wynikowy** krzywej

SK w praktyce



Przyjmujemy:

• KW = File Size

• KP = Size in memory

Q=50

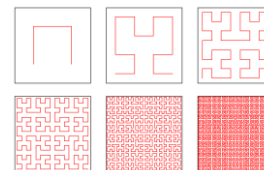
$$SK = \frac{KP}{KW} = \frac{\text{Size in memory}}{\text{File Size}} = \frac{1200\text{KB}}{31,2\text{KB}} = 38$$

Metoda RLE

- Kodowanie ciągów identycznych symboli
 - para zawiera symbol i liczbę jego powtórzeń
- Przykład kodowania:
 - 7,7,8,9,10,10,10,10,9,9,9,8,7,7,7
 - 7(2), 8(1), 9(1), 10(4), 9(3), 8(1), 7(3).
- Długość kodu zależy:
 - od rozmiaru obrazu (HxW),
 - od liczby poziomów jasności obrazu (M)

Przeszukiwanie obrazu

- Linia po linii
 - Pionowo
 - Poziomo
- Krzywa Hilberta



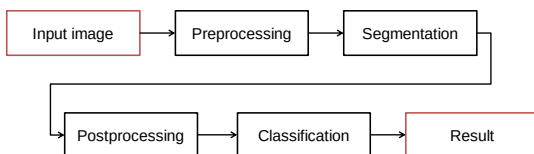
Analiza obiektów obliczanie wektora cech

Zadanie

Opracować algorytm i uruchomić aplikację realizującą wyznaczenie następujących składowych **wektora cech** obiektu binarnego:

- Momenty
- Pole powierzchni i obwód
- Współczynniki kształtu: *aspectRatio*, *extent*, *solidity*, *equivalentDiameter*

Klasyczny schemat Typowy image processing

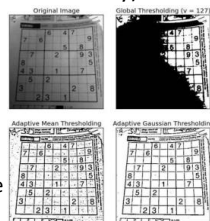


Preprocessing

- Obróbka wstępna obrazu
- Przykładowe metody:
 - Odszumianie obrazu (np. Process>Noise>Despeckle)
 - Manipulacja jasnością/kontrastem
 - Filtracja dolno/górno -przepustowa

Segmentation

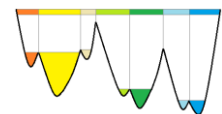
- Wyodrębnienie obiektów z obrazu
- Klasyczne podejście (przykładowe metody):
 - Progowanie (/adaptacyjne)
 - Klasteryzacja
 - Wykrywanie krawędzi
 - Rozrost regionu
- Metody oparte o AI
 - Konwulcyjne sieci neuronowe



*Image source: https://docs.opencv.org/master/d73d4d/tutorial_py_threshold.html

Postprocessing

- Obróbka już wyodrębnionych obiektów
- Przykładowe metody:
 - Dzielenie obiektów metodą wododziałową (watershed) lub metodą grafów (graph cut)
 - Zawężanie obrysu obiektów np. metodą aktywnego konturu (active contour)



*Image source: <https://media.arnov-vanity.com/render-output/4644354/figures/w1.png>

Classification

- Klasyfikacja – przypisanie obiektów do odpowiednich klas
- Na podstawie wektora cech
- Różne metody klasyfikacji:
 - Drzewa decyzyjne
 - Najbliższego sąsiada
 - Maszyna wektorów nośnych (SVM)
 - Płytki sieć neuronowa

Cechy (features)

Obrazu

- Intensity (Max/Min/Mean)
- Detected shapes
- Detected patterns

Obiektu

- Intensity (Max/Min/Mean)
- Area
- Roundness / Circularity
- Solidity
- Eccentricity
- Perimeter
- Orientation
- Euler Number

Object shape features

Parametry kształtu

$$W1 = 2 + \frac{\sqrt{S}}{\pi}$$

$$W2 = \frac{L}{\pi}$$

$$W3 = \frac{L}{2 \cdot \sqrt{\pi \cdot S}} - 1$$

$$W9 = \frac{2 \cdot \sqrt{\pi \cdot S}}{L}$$

współczynniki cykłaformości

współczynniki Malinowskiej

współczynniki Malinowskiej uproszczone

Momenty geometryczne

$$M_1 = (M_{20} + M_{02}) / m_{00}^2$$

$$M_2 = \{ (M_{20} + M_{02})^2 + 4M_{11}^2 \} / m_{00}^4$$

$$M_3 = \{ (M_{30} + 3M_{12})^2 + (3M_{21} + M_{03})^2 \} / m_{00}^6$$

$$M_4 = \{ (M_{40} + 4M_{22})^2 + (M_{31} + 3M_{13})^2 \} / m_{00}^8$$

$$M_5 = \{ M_{30} \cdot M_{03} + M_{11}^2 \} / m_{00}^5$$

$$M_6 = \{ M_{30} \cdot M_{03} + M_{21} \cdot M_{12} + M_{12}^2 + M_{21}^2 \} / m_{00}^6$$

gdzie:

S - pole ramy obiektu

L - obwód ramy obiektu

Podsumowanie Lab 4

- Segmentacja (adaptacyjna & Otsu)
- Segmentacja GrabCut
- Algorytm Watershed
- Metoda Inpainting
- Kompresja RLE
- Analiza obiektów - obliczanie wektora cech